

Experiment No.	4
Experiment Title	Binary Classification using Linear and Kernel-Based Models ¹
Student Name	Sanghamithra Namboodiri
Register Number	3122247001055
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	3rd August 2026

1 Aim and Objective

Aim: To classify emails as *spam* or *ham* using two fundamentally different families of classifiers – Logistic Regression (a linear, probabilistic model) and Support Vector Machines (a margin-based model with multiple kernel options) – and to compare their classification performance and hyperparameter sensitivity.

Objective: The aim of this experiment is to classify emails as *spam* or *ham* using two fundamentally different families of classifiers – Logistic Regression (a linear, probabilistic model) and Support Vector Machines (a margin-based model with multiple kernel options) – and to study how hyperparameter tuning affects the classification performance of each. Specifically, the experiment evaluates:

- Baseline versus tuned Logistic Regression performance.
- The behaviour of SVM across the Linear, Polynomial, RBF, and Sigmoid kernels.
- The effect of regularization strength (C) and penalty type (L1/L2) on Logistic Regression.
- The effect of kernel choice, C , γ , and polynomial degree on SVM performance.
- Generalization behaviour via 5-fold cross-validation.

2 Problem Statement

Given 57 numerical features describing word-frequency, character-frequency, and capital-run-length statistics of an email, the task is to predict the binary target **class**, where 1 denotes spam and 0 denotes ham (ordinary, non-spam email). The objective is to build and compare Logistic Regression and SVM (across four kernels) classifiers, tune their hyperparameters, and identify the model that best balances accuracy, generalization, and interpretability for spam detection.

3 Dataset Description

Dataset Name	Spambase
Dataset Source	Not specified in source document
Number of Samples	4,601 (original); 4,210 after duplicate removal; 809 after outlier removal (IQR method) – final set used for modelling

Number of Features	57 numeric features (word-frequency, character-frequency, and capital-run-length statistics)
Number of Classes	2 – binary classification (class: 0 = ham, 1 = spam)
Missing Values	None present in the raw dataset; all 58 columns had a full 4,601 non-null entries prior to cleaning
Train-Test Split	80/20 stratified split (class ratio preserved in both subsets); 5-fold cross-validation used for hyperparameter tuning

4 Exploratory Data Analysis

Include all relevant visualizations.

- Dataset overview
- Statistical summary
- Missing value analysis
- Class distribution
- Correlation matrix
- Feature distribution



Figure 1: Distribution of numeric features (word-frequency, character-frequency, and capital-run-length statistics).

Inference: Nearly every word-frequency and character-frequency feature (e.g. `word_freq_all`, `word_freq_george`, `word_freq_hpl`, `char_freq_%3B`) is extremely right-skewed, with the large majority of emails having a frequency of exactly zero for any given word or character and only a small tail of emails using that term more heavily. The `capital_run_length` features (average, longest, total) are similarly right-skewed with long tails, reflecting that most emails use short runs of capital letters while a few contain very long runs. This sparsity is exactly why the IQR-based outlier removal step (Section 5) was so aggressive on this dataset – with most values clustered at zero, any non-zero value in a sparse column tends to fall outside the IQR fence.

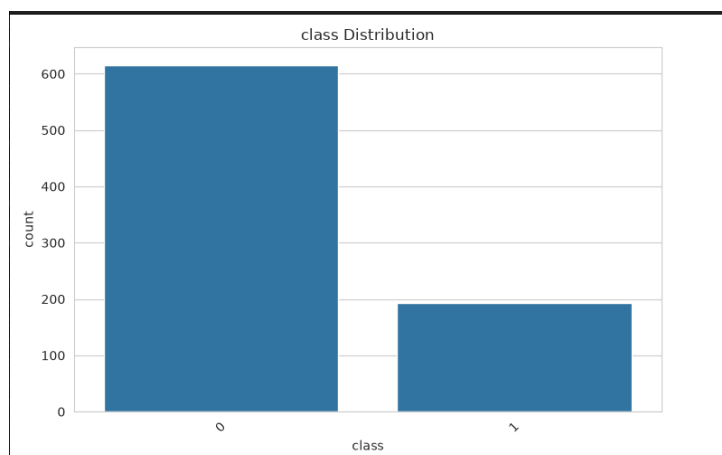


Figure 2: Class distribution of the target variable (`class`) after preprocessing.

Inference: The dataset is moderately imbalanced, with roughly 615 ham (`class=0`) emails versus about 194 spam (`class=1`) emails in the cleaned working set – a ratio of roughly 3:1. This imbalance motivated the use of a stratified train-test split to preserve the class ratio in both subsets, and is also a likely contributor to the models' comparatively higher precision/recall gap on the minority (spam) class relative to accuracy.

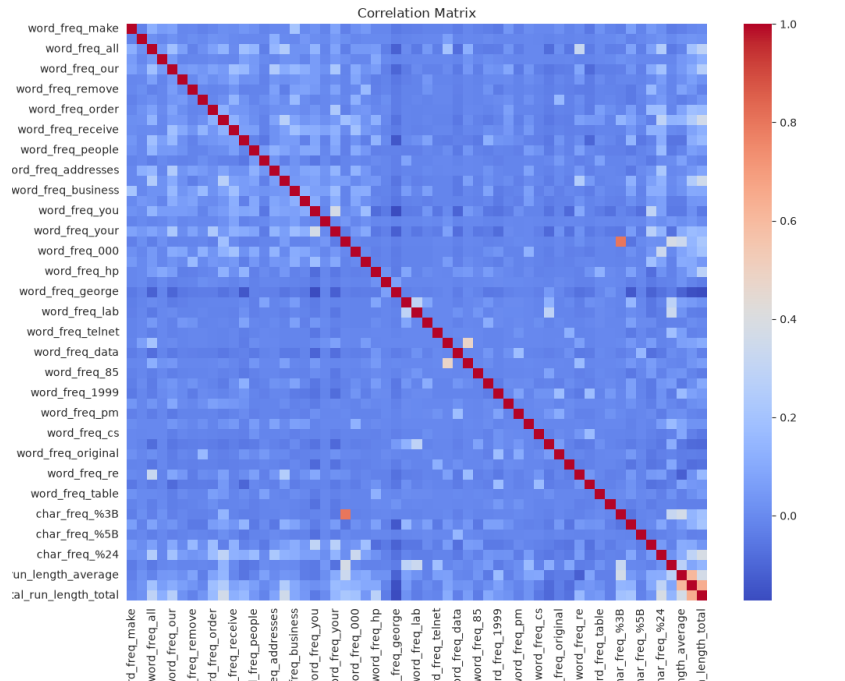


Figure 3: Correlation matrix of numeric features.

Inference: Most feature pairs show very weak correlation, as reflected by the overwhelmingly blue (near-zero) heatmap. A small number of isolated pairs – notably `word_freq_000/char_freq_%3B` and `word_freq_data/word_freq_85` – show mild positive correlation, and the `capital_run_length` features correlate somewhat with each other, as expected since they are all derived from the same capitalization statistic. The general absence of strong pairwise correlation suggests limited multicollinearity, meaning most of the 57 features contribute at least partially independent information to the classification task.

5 Data Preprocessing

1. **Missing value handling:** common placeholder strings (e.g. "NA", "?", "Unknown") were replaced with NaN and rows containing them would have been dropped; no missing values were actually present in this dataset, so no rows were removed at this step.
2. **Duplicate removal:** 391 exact duplicate rows were identified and removed, reducing the dataset from 4,601 to 4,210 rows.
3. **Outlier removal (IQR method):** for every numeric feature, values outside $[Q1 - 1.5 \text{ IQR}, Q3 + 1.5 \text{ IQR}]$ were flagged and removed. Columns with $\text{IQR} = 0$ (very sparse word-frequency columns) were skipped, since every non-zero value there would otherwise be incorrectly flagged. This step reduced the dataset from 4,210 to just 809 rows ($\sim 80\%$ reduction) – an unusually aggressive result caused by applying a single strict IQR rule independently across many skewed, sparse columns.
4. **Feature standardization:** all features were scaled using `StandardScaler` (zero

mean, unit variance) before model training, since both Logistic Regression regularization and SVM's distance-based kernels are sensitive to feature scale.

5. **Train-test split:** an 80/20 stratified split was used so that the spam/ham class ratio is preserved in both subsets.

6 Machine Learning Algorithm

Logistic Regression

Working principle: models the probability of the positive class using the logistic (sigmoid) function applied to a linear combination of features, and classifies based on a probability threshold (default 0.5).

Mathematical formulation: $P(y = 1 \mid x) = \frac{1}{1 + e^{-(\beta_0 + \sum_j \beta_j x_j)}}$, fit by maximizing the log-likelihood, optionally with an L1 or L2 penalty on β .

Advantages: fast to train, highly interpretable coefficients, outputs well-calibrated probabilities, supports L1 regularization for automatic feature sparsity.

Limitations: assumes a linear decision boundary in feature space, which limits its ability to capture non-linear class boundaries.

Support Vector Machine (SVM)

Working principle: finds the hyperplane that maximizes the margin between classes; kernels implicitly map features into higher-dimensional space to allow non-linear decision boundaries.

Mathematical formulation: minimizes $\frac{1}{2}\|w\|^2 + C \sum_i \xi_i$ subject to margin constraints, using a kernel function $K(x_i, x_j)$ (Linear, Polynomial, RBF, or Sigmoid) in place of the dot product.

Advantages: effective in high-dimensional spaces; kernel choice allows flexible modelling of linear or non-linear boundaries; robust to overfitting when the margin is well regularized.

Limitations: less interpretable than Logistic Regression, especially with non-linear kernels; more hyperparameters to tune (C , γ , kernel, degree); slower to train, particularly for tuned/non-linear kernels.

7 Experimental Setup

Table 2: Experimental setup

Item	Detail
Python Version	
Scikit-Learn Version	
IDE	
Operating System	
Hardware	

8 Hyperparameter Selection

Grid Search with 5-fold cross-validation (`GridSearchCV`, `scoring = accuracy`) was used for both models. For SVM, a separate grid was run per kernel so that kernel-specific parameters (e.g. `degree` for the polynomial kernel) were not wastefully combined with kernels that do not use them.

Table 3: Hyperparameter search spaces and best values

Parameter	Value
Logistic Regression – Penalty search space	L1, L2
Logistic Regression – C search space	{0.01, 0.1, 1, 10, 100}
Logistic Regression – Solver search space	liblinear, saga
SVM – Kernel search space	Linear, Polynomial, RBF, Sigmoid
SVM – C search space	{0.1, 1, 10, 100}
SVM – γ search space	scale, auto
SVM – Degree search space (Polynomial only)	{2, 3, 4}
Logistic Regression – Best parameters	$C = 10$, penalty = L1, solver = liblinear (Best CV Accuracy = 0.8764)
SVM – Linear (best kernel)	$C = 100$ (Best CV Accuracy = 0.8764)
SVM – Polynomial	$C = 10$, degree = 2, $\gamma = \text{scale}$
SVM – RBF	$C = 10$, $\gamma = \text{scale}$
SVM – Sigmoid	$C = 1$, $\gamma = \text{auto}$

9 Implementation Details

- **Language and core libraries:** Python, using `pandas` and `numpy` for data handling, `matplotlib/seaborn` for visualization, and `scikit-learn` for modelling (`LogisticRegression`, `SVC`, `GridSearchCV`, `StandardScaler`, `train_test_split`).
- **Pipeline structure:** the raw Spambase dataset is loaded, checked for placeholder missing-value strings, de-duplicated, IQR-filtered per numeric column (skipping zero-IQR sparse columns), standardized with `StandardScaler`, and then split 80/20 in a stratified manner to preserve the spam/ham ratio.
- **Model training:** Logistic Regression is trained with `liblinear/saga` solvers to support both L1 and L2 penalties; SVM is trained separately for each of the four kernels (Linear, Polynomial, RBF, Sigmoid) so kernel-specific hyperparameters (e.g. `degree`, γ) are tuned independently per kernel.
- **Cross-validation:** `GridSearchCV` with `cv=5` and `scoring='accuracy'` selects the best penalty/ C /solver combination for Logistic Regression and the best $C/\gamma/\text{degree}$ combination within each SVM kernel; the same 5-fold CV accuracy is reported per fold for the best Logistic Regression and best (Linear-kernel) SVM models.
- **Baseline kernel comparison:** in addition to the tuned Linear-kernel SVM, all four kernels are also fit once with default `scikit-learn` hyperparameters to obtain an untuned accuracy/F1/training-time baseline for each kernel (reported in Section 8).

- **Evaluation:** accuracy, precision, recall, F1-score, ROC-AUC, and confusion matrices are computed on the held-out 20% stratified test split ($n = 162$) for the best tuned Logistic Regression and best tuned SVM (Linear kernel) models.
- **Coefficient extraction:** the `.coef_` attribute of the tuned Logistic Regression model is extracted to rank the top-10 most influential standardized features by absolute coefficient magnitude.
- **Timing:** wall-clock training time for each model/kernel is measured with Python's `time` module, both for the tuned final models and for the untuned per-kernel SVM baselines.

10 Results

Table 4: Logistic Regression (tuned) – test-set metrics

Metric	Value
Accuracy	0.8580
Precision	0.7000
Recall	0.7179
F1-score	0.7089
ROC-AUC	0.876

Table 5: SVM, Linear kernel (tuned) – test-set metrics

Metric	Value
Accuracy	0.8148
Precision	0.6098
Recall	0.6410
F1-score	0.6250
ROC-AUC	0.861

Training time (test-set models): Logistic Regression – 0.0756s; SVM (Linear, tuned) – 0.9241s.

Table 6: SVM kernel-wise performance (default hyperparameters, untuned)

Kernel	Accuracy	F1 Score	Training Time (s)
Linear	0.8457	0.6753	0.0312
Polynomial	0.8395	0.5517	0.0211
RBF	0.8704	0.6769	0.0197
Sigmoid	0.8086	0.5507	0.0179

Table 7: Confusion matrix counts (test set, $n = 162$)

Model	TN (ham→ham)	FP (ham→spam)	FN (spam→ham)	TP (spam→spam)
Logistic Regression	111	12	11	28
SVM (Linear, tuned)	107	16	14	25

Table 8: 5-fold cross-validation accuracy

Fold	Logistic Regression	SVM
Fold 1	0.8692	0.8923
Fold 2	0.8308	0.8462
Fold 3	0.8992	0.8527
Fold 4	0.9070	0.8760
Fold 5	0.9302	0.9147
Average	0.8873	0.8764

Table 9: Top-10 Logistic Regression coefficient magnitudes

Feature	Coefficient (approx.)
word_freq_george	12.4
word_freq_hpl	9.3
word_freq_project	4.9
word_freq_lab	4.5
word_freq_meeting	4.2
word_freq_edu	3.4
word_freq_data	3.2
word_freq_business	1.9
char_freq_%3B	1.85
word_freq_addresses	1.8

Table 10: Comparative summary: Logistic Regression vs. SVM

Criterion	Logistic Regression	SVM
Accuracy (test set)	0.8580	0.8148
ROC-AUC (test set)	0.876	0.861
Training Time (s)	0.0756	0.9241
Model Complexity	Low	High
Interpretability	High	Low

11 Result Visualization

Include all graphs generated during the experiment. For every graph, provide a figure, caption, and inference.

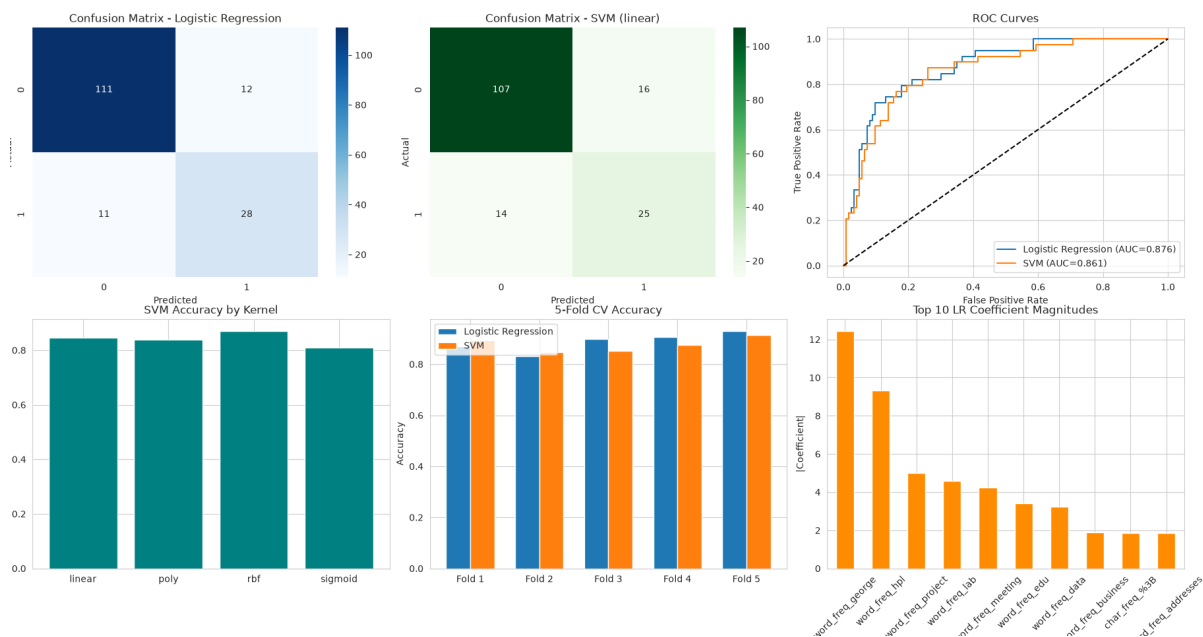


Figure 4: Combined results dashboard: (a) confusion matrix – Logistic Regression, (b) confusion matrix – SVM (Linear kernel), (c) ROC curves, (d) SVM accuracy by kernel, (e) 5-fold CV accuracy per fold, (f) top-10 Logistic Regression coefficient magnitudes.

Inference: Panel (a) shows the tuned Logistic Regression model correctly classified 111 ham and 28 spam emails out of 162, with 12 false positives and 11 false negatives, matching its Accuracy (0.8580), Precision (0.7000), Recall (0.7179), and F1 (0.7089). Panel (b) shows the best (tuned) SVM correctly classified 107 ham and 25 spam emails, with 16 false positives and 14 false negatives – slightly more errors on both sides than Logistic Regression, consistent with its lower test Accuracy (0.8148) and F1 (0.6250). Panel (c)’s ROC curves both sit well above the diagonal, confirming strong discriminative ability; Logistic Regression attains a marginally higher AUC (0.876) than SVM (0.861). Panel (d) compares the four untuned SVM kernels: RBF achieves the highest baseline accuracy (0.8704), followed by Linear (0.8457) and Polynomial (0.8395), with Sigmoid clearly the weakest (0.8086) – suggesting the spam/ham boundary is closer to linear or mildly non-linear than to the shape a sigmoid kernel imposes. Panel (e) shows both models tracking each other closely fold-to-fold across the 5-fold cross-validation, with accuracy rising in later folds for both and no single collapsing fold, suggesting a reasonably stable CV estimate despite the small sample size remaining after outlier removal. Panel (f) shows `word_freq_george` and `word_freq_hpl` dominate the top-10 Logistic Regression coefficients by a wide margin, followed by `word_freq_project`, `word_freq_lab`, and `word_freq_meeting` – workplace/organization-specific terms (an artifact of the dataset’s source mailbox) that strongly indicate *ham* when present.

12 Overfitting and Underfitting Analysis

Logistic Regression achieved a best 5-fold cross-validation accuracy of 0.8873, but its held-out test accuracy was slightly lower at 0.8580 – a gap of about 3 percentage points. The best SVM kernel (Linear) reached 0.8764 average CV accuracy but only 0.8148 on the test set, a larger gap of roughly 6 percentage points. This indicates mild overfitting to the training folds in both models, more pronounced in SVM than in Logistic Regression.

For Logistic Regression, the best model selected L1 regularization with $C = 10$, i.e. relatively weak regularization – the model prefers to retain most of its predictive coefficients rather than shrinking them aggressively, consistent with Spambase’s features already being fairly informative and non-redundant after standardization. For SVM, the winning linear kernel also selected the largest tested C value (100), similarly implying weak regularization is preferred – the model is allowed a narrower margin in exchange for fitting the training data more closely, which is one explanation for its larger train–test performance gap.

Comparing baseline SVM kernel accuracies to their tuned counterparts shows the RBF kernel actually performed best untuned (0.8704) but the Linear kernel edged ahead after tuning based on cross-validation accuracy (0.8764), underscoring that grid search over $C/\gamma/\text{degree}$ meaningfully shifts which kernel is “best” – default hyperparameters do not always identify the optimal kernel. With only 809 training rows surviving pre-processing (from an original 4,210), both models were trained on a comparatively small, class-imbalanced sample, raising the risk of overfitting to idiosyncrasies of that specific subset and likely contributing to the CV–test accuracy gap.

13 Bias–Variance Analysis

As a linear model, Logistic Regression assumes a linear decision boundary in the (standardized) feature space. This constrains its ability to capture non-linear spam/ham boundaries (higher bias) but makes it less sensitive to noise in the small training set (lower variance), reflected in its smaller CV–test accuracy gap (~ 3 points) compared to SVM.

The margin-based SVM’s bias–variance behaviour depends heavily on the chosen kernel: the Linear kernel behaves similarly to Logistic Regression (higher bias, lower variance) and was ultimately selected as best after tuning; the RBF kernel can model highly non-linear boundaries (lower bias) but is more prone to overfitting small, noisy training sets (higher variance), which may explain why its untuned performance (0.8704) was competitive with the tuned Linear kernel despite less tuning effort; the Polynomial and Sigmoid kernels showed the weakest performance and lowest F1 scores among the four, suggesting they are either poorly suited to this feature space or highly sensitive to the specific hyperparameters chosen.

For both models, C (inverse regularization strength) directly trades bias for variance: smaller C increases bias (simpler decision boundary, more regularization) while larger C increases variance (tighter fit to training data). The fact that both winning models chose relatively large C values (10 and 100) suggests that, on this particular (heavily outlier-filtered) training sample, additional regularization was not needed to control variance – though the CV–test gap suggests a smaller C might improve generalization on unseen data.

14 Observations and Conclusion

Observations

- Logistic Regression outperforms the best-tuned SVM on every held-out test metric (Accuracy, Precision, Recall, F1, ROC-AUC) despite SVM achieving a comparable

5-fold CV accuracy, indicating SVM generalizes less reliably from training folds to unseen data on this dataset.

- Both models show a CV-to-test accuracy gap (Logistic Regression ~ 3 points, SVM ~ 6 points), consistent with mild overfitting that is more pronounced for SVM.
- Among the four SVM kernels, RBF and Linear are clearly the strongest (untuned accuracies of 0.8704 and 0.8457), while Polynomial and especially Sigmoid lag behind, suggesting the underlying spam/ham boundary is close to linear or mildly non-linear.
- Grid search shifts the ranking of "best" SVM kernel: RBF wins untuned, but Linear wins after tuning C , showing that default hyperparameters do not reliably identify the best kernel.
- Both winning models (Logistic Regression and SVM-Linear) selected large regularization/ C values, implying the models preferred to fit the training data closely rather than shrink aggressively – consistent with informative, largely non-redundant standardized features.
- Logistic Regression is dramatically faster to train (0.0756s vs. 0.9241s for SVM) and far more interpretable, since its coefficients directly rank feature importance.
- The top Logistic Regression coefficients (`word_freq_george`, `word_freq_hpl`, `word_freq_project`, `word_freq_lab`, `word_freq_meeting`) are workplace/mailbox-specific terms rather than generic spam indicators – a known artifact of the Spambase corpus's source mailbox.
- The aggressive IQR-based outlier removal step discarded roughly 80% of the de-duplicated dataset ($4,210 \rightarrow 809$ rows), which likely constrained both models' ability to learn robust decision boundaries and contributed to the CV-test performance gap.

Conclusion

- **Best-performing classifier:** Logistic Regression outperformed the best-tuned SVM on the held-out test set (Accuracy 0.8580 vs. 0.8148; F1 Score 0.7089 vs. 0.6250; AUC 0.876 vs. 0.861), despite SVM achieving a comparable cross-validation accuracy. Logistic Regression also trained roughly $12\times$ faster (0.076s vs. 0.924s).
- **Impact of regularization:** L1 regularization was preferred for Logistic Regression, indicating some benefit from feature sparsity, though the selected $C = 10$ shows the model did not need heavy shrinkage to generalize reasonably well on this dataset.
- **Kernel behaviour:** Among SVM kernels, Linear and RBF were clearly the strongest performers, while Polynomial and Sigmoid lagged behind – consistent with Spambase's features being largely linearly separable after standardization, with RBF adding modest non-linear flexibility.
- **Bias-variance trade-off:** Logistic Regression's simplicity (higher bias, lower variance) generalized more reliably than SVM's more flexible margin-based decision

boundary on this small, outlier-filtered dataset, where variance control matters more than raw model expressiveness.

- **Feature importance:** The dominant Logistic Regression coefficients (`word_freq_george`, `word_freq_hpl`, `word_freq_project`, `word_freq_lab`, `word_freq_meeting`) are workplace-mailbox-specific terms rather than generic spam indicators, a known quirk of the Spambase corpus that limits how well this particular feature ranking would generalize to spam filtering on a different mailbox.
- **Overall conclusion:** For this spam-classification task, Logistic Regression with L1 regularization ($C = 10$) is the recommended model – it is more accurate on unseen data, dramatically faster to train, and considerably more interpretable, making it preferable to SVM both practically and analytically for this use case.
- **Limitation:** The IQR-based outlier removal step discarded roughly 80% of the cleaned dataset ($4,210 \rightarrow 809$ rows), which likely limited both models' ability to generalize. A less aggressive or feature-aware outlier-handling strategy (e.g. skipping IQR filtering entirely for sparse count-like features) would be a natural next step to improve results in a future iteration of this experiment.

15 References

1. Spambase Dataset – 4,601 email records with 57 numeric word-frequency, character-frequency, and capital-run-length features.
2. F. Pedregosa *et al.*, "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
3. C. Cortes and V. Vapnik, "Support-Vector Networks," *Machine Learning*, vol. 20, pp. 273–297, 1995.